Java Important Exam Questions & Answers

Complete 5-Mark Answers

Q1. What are the features of Java HashMap?

HashMap is a part of Java's Collection Framework (java.util package) that stores data as key-value pairs using hashing.

Key Features:

- Stores data as key-value pairs (Key must be unique; values can be duplicate).
- Allows one null key and multiple null values.
- Non-synchronized — not thread-safe by default (use ConcurrentHashMap for thread safety).
- No guaranteed insertion order (use LinkedHashMap for order).
- Backed by a hash table; provides O(1) average time for get/put operations.
- Initial capacity is 16 with a load factor of 0.75 (resizes when 75% full).

```
import java.util.HashMap;
HashMap<String, Integer> map = new HashMap<>();
map.put("Alice", 90);
map.put("Bob", 85);
map.put(null, 70);       // null key allowed
System.out.println(map.get("Alice")); // 90
System.out.println(map.containsKey("Bob")); // true
```

Q2. Differentiate between ArrayList and LinkedList.

Both implement the List interface but differ in internal structure and performance:

```
Feature          | ArrayList              | LinkedList
-----------------+------------------------+------------------------
Structure        | Dynamic array          | Doubly linked list
Random Access    | O(1) - fast            | O(n) - slow
Insert/Delete    | O(n) - slow (shifting) | O(1) at known node
Memory           | Less (only data)       | More (data + 2 pointers)
Best Use         | Read-heavy apps        | Insert/delete-heavy apps
Implements       | List                   | List, Queue, Deque
```

```
ArrayList<Integer> al = new ArrayList<>();
al.add(10); al.add(20); al.get(0); // fast

LinkedList<Integer> ll = new LinkedList<>();
ll.add(10); ll.addFirst(5); // efficient at ends
```

Q3. What is Collection Framework in Java? Explain with example.

The Java Collection Framework (JCF) is a unified architecture for storing and manipulating groups of objects. It provides interfaces, implementations, and algorithms.

Core Interfaces: Collection, List, Set, Queue, Map

Common Implementations:

- List: ArrayList, LinkedList, Vector
- Set: HashSet, TreeSet, LinkedHashSet

- Queue: PriorityQueue, ArrayDeque
- Map: HashMap, TreeMap, LinkedHashMap

```
import java.util.*;
List<String> list = new ArrayList<>();
list.add("Java"); list.add("Python"); list.add("C++");
Collections.sort(list);
for(String s : list) System.out.println(s);

Set<Integer> set = new HashSet<>();
set.add(1); set.add(2); set.add(2); // duplicate ignored
System.out.println(set); // [1, 2]

Map<String,Integer> map = new HashMap<>();
map.put("marks", 95);
System.out.println(map.get("marks")); // 95
```

Q4. What are the main differences between Array and Collection?

Feature	Array	Collection
Size	Fixed at creation	Dynamic (grows/shrinks)
Type	Primitive + Objects	Objects only
Performance	Faster (no overhead)	Slightly slower
Methods	No built-in utility	Rich API (sort, search, etc.)
Generics	Not supported	Fully supported
Null values	Allowed	Depends on implementation
Homogeneous	Yes (one type)	Yes (with generics)

```
int[] arr = new int[5]; // fixed size
arr[0] = 10;

ArrayList<Integer> col = new ArrayList<>();
col.add(10); col.add(20); // dynamic
```

Q5. Explain EventListener with example.

An EventListener is a marker interface in java.util that is extended by all event listener interfaces. In GUI programming (AWT/Swing), event listeners respond to user actions like button clicks, mouse movement, key presses, etc.

Common EventListeners: ActionListener, MouseListener, KeyListener, WindowListener

Steps: 1) Create a component 2) Implement a listener interface 3) Register listener with addXxxListener()

```
import java.awt.*;
import java.awt.event.*;

public class EventDemo extends Frame implements ActionListener {
    Button btn;
    Label lbl;

    EventDemo() {
        btn = new Button("Click Me");
        lbl = new Label("Hello");
        btn.addActionListener(this); // register listener
        add(btn); add(lbl);
        setSize(300, 200); setVisible(true);
    }
```

```
    public void actionPerformed(ActionEvent e) {
        lbl.setText("Button Clicked!");
    }

    public static void main(String[] args) { new EventDemo(); }
}
```

Q6. Explain ActionListener Interface with example.

ActionListener is a functional interface in java.awt.event that handles action events (e.g., button clicks). It has one method: actionPerformed(ActionEvent e).

```
import javax.swing.*;
import java.awt.event.*;

public class ActionDemo extends JFrame {
    JButton btn = new JButton("Greet");
    JLabel lbl = new JLabel("---");

    ActionDemo() {
        setLayout(new java.awt.FlowLayout());
        add(btn); add(lbl);

        btn.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                lbl.setText("Hello, World!");
            }
        });

        // Lambda alternative (Java 8+):
        // btn.addActionListener(e -> lbl.setText("Hello!"));

        setSize(300, 150); setVisible(true); setDefaultCloseOperation(EXIT_ON_CLOSE);
    }

    public static void main(String[] args) { new ActionDemo(); }
}
```

Q7. Differences between creating a thread by extending Thread class and implementing Runnable interface.

```
Feature          | Extending Thread        | Implementing Runnable
-----------------+-------------------------+-----------------------
Inheritance      | Cannot extend other     | Can extend another class
                 | class (Java single      |
                 | inheritance)            |
Reusability      | Less reusable           | More reusable (can pass
                 |                         | to Executor, Thread, etc.)
Code coupling    | Tightly coupled         | Loosely coupled
Starting         | t.start()               | new Thread(r).start()
Preferred        | Rare cases              | Recommended approach
```

```
// Method 1: Extending Thread
class MyThread extends Thread {
    public void run() { System.out.println("Thread running"); }
}
new MyThread().start();

// Method 2: Implementing Runnable
```

```
class MyRunnable implements Runnable {
    public void run() { System.out.println("Runnable running"); }
}
new Thread(new MyRunnable()).start();
// Or with lambda:
new Thread(() -> System.out.println("Lambda thread")).start();
```

Q8. Write a Java program where multiple threads perform multiple tasks.

```
class DownloadTask implements Runnable {
    String file;
    DownloadTask(String f) { file = f; }
    public void run() {
        System.out.println(Thread.currentThread().getName()
            + " downloading: " + file);
        try { Thread.sleep(1000); } catch(InterruptedException e) {}
        System.out.println(file + " downloaded.");
    }
}

class PrintTask implements Runnable {
    public void run() {
        for(int i = 1; i <= 5; i++) {
            System.out.println("Printing page " + i);
            try { Thread.sleep(500); } catch(InterruptedException e) {}
        }
    }
}

public class MultiTaskDemo {
    public static void main(String[] args) {
        Thread t1 = new Thread(new DownloadTask("Report.pdf"), "Downloader-1");
        Thread t2 = new Thread(new DownloadTask("Image.png"), "Downloader-2");
        Thread t3 = new Thread(new PrintTask(), "Printer");
        t1.start(); t2.start(); t3.start();
    }
}
```

Q9. What are the advantages of multithreading?

- Better CPU Utilization: Multiple threads keep the CPU busy, reducing idle time.
- Improved Performance: Tasks run concurrently, reducing total execution time.
- Responsiveness: UI applications remain responsive while background tasks run.
- Resource Sharing: Threads share the same memory and resources of the process.
- Parallelism: On multi-core CPUs, threads run truly simultaneously.
- Simpler Modeling: Some problems (producer-consumer, servers) naturally map to threads.
- Asynchronous Operations: I/O-bound tasks (file, network) don't block the main thread.

Example: A web server handles thousands of client requests using separate threads, allowing simultaneous service without blocking.

Q10. Differentiate between sleep() and wait() method.

```
Feature         | sleep()                    | wait()
----------------+----------------------------+-----------------------------
Package         | java.lang.Thread           | java.lang.Object
```

```
Synchronization | Not required               | Must be in synchronized block
Lock release    | Does NOT release lock      | RELEASES the lock
Wake up         | After timeout (automatic)  | notify() / notifyAll() / timeout
Purpose         | Pause execution for time   | Inter-thread communication
Exception       | InterruptedException       | InterruptedException
```

```
// sleep() example
Thread.sleep(1000); // pauses for 1 second, keeps lock

// wait() example
synchronized(obj) {
    obj.wait();    // releases lock, waits for notify
}
// In another thread:
synchronized(obj) {
    obj.notify();  // wakes up waiting thread
}
```

Q11. Write a Java program where main thread and child thread run simultaneously.

```
class ChildThread extends Thread {
    public void run() {
        for(int i = 1; i <= 5; i++) {
            System.out.println("Child Thread: " + i);
            try { Thread.sleep(500); } catch(InterruptedException e) {}
        }
        System.out.println("Child Thread finished.");
    }
}

public class MainChildDemo {
    public static void main(String[] args) throws InterruptedException {
        ChildThread child = new ChildThread();
        child.start(); // child starts running

        for(int i = 1; i <= 5; i++) {
            System.out.println("Main Thread: " + i);
            Thread.sleep(500);
        }
        System.out.println("Main Thread finished.");
    }
}
// Output: Main Thread and Child Thread lines interleave.
```

Q12. Write a Java program to handle multiple exceptions using nested try block.

```
public class NestedTryDemo {
    public static void main(String[] args) {
        try {
            System.out.println("Outer try block");
            int[] arr = {10, 20, 30};

            try {
                System.out.println("Inner try block");
                int result = arr[1] / 0; // ArithmeticException
            } catch(ArithmeticException e) {
                System.out.println("Inner catch: " + e.getMessage());
            }
```

```
            int x = arr[5]; // ArrayIndexOutOfBoundsException

        } catch(ArrayIndexOutOfBoundsException e) {
            System.out.println("Outer catch: " + e.getMessage());
        } finally {
            System.out.println("Finally block always executes.");
        }
    }
}
```

Q13. Write a program to calculate percentage of a student for 3 subjects and create exception if marks > 100.

```
class InvalidMarksException extends Exception {
    InvalidMarksException(String msg) { super(msg); }
}

public class StudentPercentage {
    static double calcPercentage(int m1, int m2, int m3)
            throws InvalidMarksException {
        if(m1 > 100 || m2 > 100 || m3 > 100)
            throw new InvalidMarksException("Marks cannot exceed 100!");
        if(m1 < 0 || m2 < 0 || m3 < 0)
            throw new InvalidMarksException("Marks cannot be negative!");
        return (m1 + m2 + m3) / 3.0;
    }

    public static void main(String[] args) {
        try {
            System.out.printf("Percentage: %.2f%%
", calcPercentage(85, 90, 78));
            System.out.printf("Percentage: %.2f%%
", calcPercentage(85, 110, 78)); // throws
        } catch(InvalidMarksException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Q14. Explain life cycle of a thread with diagram.

A thread goes through 5 states during its lifetime:

```
  [NEW] --start()--> [RUNNABLE] --scheduler--> [RUNNING]
                          ^                        |
                          |     sleep()/wait()     |
                          +----[BLOCKED/WAITING]<--+
                                                   |
                                              run() ends
                                                   |
                                             [TERMINATED]
```

- **New:** Thread object created but start() not yet called.
- **Runnable:** start() called; thread is ready to run, waiting for CPU.
- **Running:** Thread scheduler selects it; run() executes.
- **Blocked/Waiting:** Thread waits for I/O, sleep(), wait(), or lock acquisition.
- **Terminated (Dead):** run() method completes or stop() is called.

Key methods that cause state transitions: start(), sleep(), wait(), notify(), join(), yield().

Q15. Show different types of exceptions in Java (negative number exception).

Java exceptions are divided into: **Checked** (compile-time) and **Unchecked** (runtime).

```
Types of Exceptions:
Throwable
 +-- Error (JVM errors: StackOverflowError, OutOfMemoryError)
 +-- Exception
      +-- Checked: IOException, SQLException, ClassNotFoundException
      +-- Unchecked (RuntimeException):
           ArithmeticException, NullPointerException,
           ArrayIndexOutOfBoundsException, NumberFormatException
```

```
// Custom: Negative Number Exception
class NegativeNumberException extends Exception {
    NegativeNumberException(int n) {
        super("Negative number not allowed: " + n);
    }
}

public class ExceptionDemo {
    static int squareRoot(int n) throws NegativeNumberException {
        if(n < 0) throw new NegativeNumberException(n);
        return (int) Math.sqrt(n);
    }
    public static void main(String[] args) {
        try {
            System.out.println(squareRoot(16));  // 4
            System.out.println(squareRoot(-9));  // throws
        } catch(NegativeNumberException e) {
            System.out.println("Caught: " + e.getMessage());
        }
    }
}
```

Q16. Write a program to calculate volume of a sphere and cone using interface Shape.

```
interface Shape {
    double volume();
    double PI = 3.14159265;
}

class Sphere implements Shape {
    double radius;
    Sphere(double r) { radius = r; }
    public double volume() {
        return (4.0/3.0) * PI * radius * radius * radius;
    }
}

class Cone implements Shape {
    double radius, height;
    Cone(double r, double h) { radius = r; height = h; }
    public double volume() {
        return (1.0/3.0) * PI * radius * radius * height;
    }
```

```
}

public class ShapeDemo {
    public static void main(String[] args) {
        Shape s = new Sphere(5);
        System.out.printf("Sphere Volume: %.2f
", s.volume());

        Shape c = new Cone(4, 9);
        System.out.printf("Cone Volume: %.2f
", c.volume());
    }
}
```

Q17. Explain the use of final keyword in Java.

The **final** keyword is used in three contexts:

1. Final Variable — value cannot be changed (constant):

```
final int MAX = 100;
MAX = 200; // Compile error!
```

2. Final Method — cannot be overridden by subclass:

```
class Parent {
    final void display() { System.out.println("Parent"); }
}
class Child extends Parent {
    void display() { } // Compile error!
}
```

3. Final Class — cannot be subclassed (e.g., String, Integer):

```
final class MyClass { }
class Sub extends MyClass { } // Compile error!
```

Final variables must be initialized at declaration or in the constructor. Used to create constants, prevent inheritance, and improve performance via inlining.

Q18. Write a Java program to create user defined package and subpackage.

```
// File: mypack/greeting/Hello.java
package mypack.greeting;
public class Hello {
    public void greet(String name) {
        System.out.println("Hello, " + name + "!");
    }
}

// File: mypack/math/Calculator.java
package mypack.math;
public class Calculator {
    public int add(int a, int b) { return a + b; }
}

// File: Main.java (in root directory)
import mypack.greeting.Hello;
import mypack.math.Calculator;

public class Main {
    public static void main(String[] args) {
```

```
        Hello h = new Hello();
        h.greet("Anik");

        Calculator c = new Calculator();
        System.out.println("Sum = " + c.add(10, 20));
    }
}

// Compile: javac mypack/greeting/Hello.java
//          javac mypack/math/Calculator.java
//          javac Main.java
// Run:     java Main
```

Q19. Explain Dynamic Method Dispatch with example.

Dynamic Method Dispatch (also called Runtime Polymorphism) is the mechanism by which a call to an overridden method is resolved at **runtime** rather than compile-time. It is achieved through method overriding and a parent reference pointing to a child object.

```
class Animal {
    void sound() { System.out.println("Animal makes a sound"); }
}
class Dog extends Animal {
    void sound() { System.out.println("Dog barks"); }
}
class Cat extends Animal {
    void sound() { System.out.println("Cat meows"); }
}

public class DispatchDemo {
    public static void main(String[] args) {
        Animal a;           // parent reference

        a = new Dog();      // Dog object
        a.sound();          // OUTPUT: Dog barks

        a = new Cat();      // Cat object
        a.sound();          // OUTPUT: Cat meows

        a = new Animal();
        a.sound();          // OUTPUT: Animal makes a sound
    }
}
```

The JVM checks the actual object type at runtime to decide which method to call — this is the core of runtime polymorphism.

Q20. Write an interface program to calculate child height from parents' height.

A common formula: Child Height = (Father's Height + Mother's Height) / 2 (+6.5 cm for boy, -6.5 cm for girl)

```
interface HeightPredictor {
    double predictHeight(double fatherHeight, double motherHeight);
}

class BoyHeight implements HeightPredictor {
    public double predictHeight(double f, double m) {
        return (f + m) / 2 + 6.5;
```

```
    }
}

class GirlHeight implements HeightPredictor {
    public double predictHeight(double f, double m) {
        return (f + m) / 2 - 6.5;
    }
}

public class ChildHeightDemo {
    public static void main(String[] args) {
        double father = 175.0, mother = 162.0;

        HeightPredictor boy = new BoyHeight();
        HeightPredictor girl = new GirlHeight();

        System.out.printf("Predicted Boy Height:  %.1f cm
",
            boy.predictHeight(father, mother));
        System.out.printf("Predicted Girl Height: %.1f cm
",
            girl.predictHeight(father, mother));
    }
}
```

Q21. Explain super keyword with example.

The **super** keyword refers to the immediate parent class. It has three main uses:

1. Access parent class variable (when hidden by child variable):

```
class Animal { String name = "Animal"; }
class Dog extends Animal {
    String name = "Dog";
    void display() {
        System.out.println(super.name); // Animal
        System.out.println(this.name);  // Dog
    }
}
```

2. Call parent class method (when overridden):

```
class Animal { void sound() { System.out.println("Generic sound"); } }
class Dog extends Animal {
    void sound() {
        super.sound(); // calls parent method
        System.out.println("Bark");
    }
}
```

3. Call parent class constructor:

```
class Animal {
    Animal(String name) { System.out.println("Animal: " + name); }
}
class Dog extends Animal {
    Dog(String name) {
        super(name); // must be first statement
        System.out.println("Dog created");
    }
}
```

Q22. Explain constructor inheritance with example.

Constructors are **NOT inherited** in Java. However, when a child class object is created, the parent constructor is automatically called via **super()** (implicitly or explicitly). This ensures proper initialization of the parent part of the object.

```
class Vehicle {
    String type;
    Vehicle(String t) {
        type = t;
        System.out.println("Vehicle constructor called: " + t);
    }
}

class Car extends Vehicle {
    String model;
    Car(String model) {
        super("Car");  // explicitly calling parent constructor
        this.model = model;
        System.out.println("Car constructor called: " + model);
    }
}

class ElectricCar extends Car {
    ElectricCar() {
        super("Tesla");  // calls Car constructor
        System.out.println("ElectricCar constructor called");
    }
}

public class ConstructorDemo {
    public static void main(String[] args) {
        ElectricCar ec = new ElectricCar();
        // Output (top-down initialization):
        // Vehicle constructor called: Car
        // Car constructor called: Tesla
        // ElectricCar constructor called
    }
}
```

Q23. Write a program to find area of square, rectangle and triangle using parameterized constructor.

```
class Square {
    double side;
    Square(double s) { side = s; }
    double area() { return side * side; }
}

class Rectangle {
    double length, width;
    Rectangle(double l, double w) { length = l; width = w; }
    double area() { return length * width; }
}

class Triangle {
    double base, height;
    Triangle(double b, double h) { base = b; height = h; }
```

```
    double area() { return 0.5 * base * height; }
}

public class AreaDemo {
    public static void main(String[] args) {
        Square sq = new Square(5);
        Rectangle rect = new Rectangle(8, 4);
        Triangle tri = new Triangle(6, 9);

        System.out.println("Area of Square:    " + sq.area());
        System.out.println("Area of Rectangle: " + rect.area());
        System.out.println("Area of Triangle:  " + tri.area());
    }
}
```

Q24. Explain Array of Objects with example.

An array of objects stores references to multiple objects of the same class. Each element in the array is a reference to an object created using new.

```
class Student {
    String name;
    int marks;

    Student(String name, int marks) {
        this.name = name;
        this.marks = marks;
    }

    void display() {
        System.out.println("Name: " + name + ", Marks: " + marks);
    }
}

public class ArrayOfObjects {
    public static void main(String[] args) {
        // Declare array of Student objects (size 3)
        Student[] students = new Student[3];

        // Initialize each object
        students[0] = new Student("Alice", 90);
        students[1] = new Student("Bob",   85);
        students[2] = new Student("Carol", 92);

        // Access objects
        for(Student s : students) {
            s.display();
        }

        // Find topper
        Student topper = students[0];
        for(Student s : students)
            if(s.marks > topper.marks) topper = s;
        System.out.println("Topper: " + topper.name);
    }
}
```

Q25. Write a program to compare two strings using ==, compareTo() and equals() method.

```
public class StringCompare {
    public static void main(String[] args) {
        String s1 = new String("Hello");
        String s2 = new String("Hello");
        String s3 = s1;

        // 1. == operator: compares references (memory addresses)
        System.out.println("== (s1,s2): " + (s1 == s2)); // false (diff objects)
        System.out.println("== (s1,s3): " + (s1 == s3)); // true (same reference)

        // 2. equals(): compares content (characters)
        System.out.println("equals(s1,s2): " + s1.equals(s2));       // true
        System.out.println("equalsIgnoreCase: " + s1.equalsIgnoreCase("hello")); // true

        // 3. compareTo(): lexicographic comparison
        // Returns 0 if equal, <0 if s1<s2, >0 if s1>s2
        String a = "Apple", b = "Banana", c = "Apple";
        System.out.println("compareTo(a,b): " + a.compareTo(b)); // negative
        System.out.println("compareTo(a,c): " + a.compareTo(c)); // 0 (equal)
        System.out.println("compareTo(b,a): " + b.compareTo(a)); // positive
    }
}
```

Q26. Explain Polymorphism with example.

Polymorphism means 'many forms'. It allows one interface/method to work differently for different types. Java supports two types:

1. Compile-time Polymorphism (Method Overloading):

```
class Calculator {
    int add(int a, int b)           { return a + b; }
    double add(double a, double b)  { return a + b; }
    int add(int a, int b, int c)    { return a + b + c; }
}
// Same name, different parameters — resolved at compile time
```

2. Runtime Polymorphism (Method Overriding):

```
class Shape { void draw() { System.out.println("Drawing shape"); } }
class Circle extends Shape { void draw() { System.out.println("Drawing circle"); } }
class Rect   extends Shape { void draw() { System.out.println("Drawing rectangle"); } }

public class PolyDemo {
    public static void main(String[] args) {
        Shape[] shapes = { new Circle(), new Rect(), new Shape() };
        for(Shape s : shapes)
            s.draw(); // resolved at runtime — polymorphic call
    }
}
```

Q27. Write a program to find higher salary using OOP approach and accessor method.

```
class Employee {
    private String name;
```

```
    private double salary;

    // Constructor
    Employee(String name, double salary) {
        this.name = name;
        this.salary = salary;
    }

    // Accessor (getter) methods
    public String getName()   { return name; }
    public double getSalary() { return salary; }

    // Mutator (setter) methods
    public void setSalary(double s) { salary = s; }

    public static Employee higherSalary(Employee e1, Employee e2) {
        return (e1.getSalary() >= e2.getSalary()) ? e1 : e2;
    }
}

public class SalaryDemo {
    public static void main(String[] args) {
        Employee e1 = new Employee("Alice", 75000);
        Employee e2 = new Employee("Bob",   82000);

        Employee top = Employee.higherSalary(e1, e2);
        System.out.println("Higher salary: " + top.getName()
            + " -> Rs. " + top.getSalary());
    }
}
```

Q28. Explain static keyword and its uses.

The **static** keyword indicates that a member belongs to the **class**, not to individual objects. It is shared across all instances.

Uses of static:

- **Static Variable:** Shared among all objects (class-level variable).
- **Static Method:** Can be called without creating an object.
- **Static Block:** Executes once when the class is loaded.
- **Static Nested Class:** Nested class that doesn't need an outer instance.

```
class Counter {
    static int count = 0; // shared

    static { System.out.println("Class loaded!"); } // static block

    Counter() { count++; }

    static int getCount() { return count; } // static method
}

public class StaticDemo {
    public static void main(String[] args) {
        new Counter(); new Counter(); new Counter();
        System.out.println("Objects: " + Counter.getCount()); // 3
        // No object needed to call static method
    }
```

```
}
```

Q29. What is Event Handling in Java?

Event Handling is the mechanism that controls events (user actions) and decides what should happen when an event occurs. Java uses a **Delegation Event Model**.

Key Components:

- **Event Source:** The object that generates the event (e.g., Button, TextField).
- **Event Object:** Encapsulates information about the event (e.g., ActionEvent, MouseEvent).
- **Event Listener:** Interface that receives and handles the event.

Steps in Event Handling:

- 1. User performs an action (click, type, move mouse).
- 2. An event object is created by the source.
- 3. The event is dispatched to the registered listener.
- 4. The listener's method is invoked to handle the event.

```
// Common listeners and their methods:
ActionListener  -> actionPerformed(ActionEvent e)
MouseListener   -> mouseClicked(), mousePressed(), mouseReleased()...
KeyListener     -> keyPressed(), keyReleased(), keyTyped()
WindowListener  -> windowClosing(), windowOpened()...
```

Q30. Explain MouseListener with example.

MouseListener is an interface in java.awt.event that handles 5 mouse events: mouseClicked, mouseEntered, mouseExited, mousePressed, mouseReleased.

```
import java.awt.*;
import java.awt.event.*;

public class MouseDemo extends Frame implements MouseListener {
    Label lbl;

    MouseDemo() {
        lbl = new Label("Move/click the mouse here");
        add(lbl);
        addMouseListener(this); // register listener
        setSize(400, 300); setVisible(true);
    }

    public void mouseClicked(MouseEvent e) {
        lbl.setText("Clicked at (" + e.getX() + ", " + e.getY() + ")");
    }
    public void mouseEntered(MouseEvent e) { lbl.setText("Mouse Entered"); }
    public void mouseExited(MouseEvent e)  { lbl.setText("Mouse Exited");  }
    public void mousePressed(MouseEvent e) { lbl.setText("Mouse Pressed"); }
    public void mouseReleased(MouseEvent e){ lbl.setText("Mouse Released");}

    public static void main(String[] args) { new MouseDemo(); }
}
```

Q31. Explain Java Collection Framework and state four main types of collection interfaces.

The Java Collection Framework (JCF) provides a standard architecture for storing and manipulating groups of objects using interfaces, implementations, and algorithms.

Four Main Interfaces:

- **List** — Ordered, allows duplicates. Implementations: ArrayList, LinkedList, Vector. Example: a todo list.
- **Set** — Unordered (or sorted), NO duplicates. Implementations: HashSet, TreeSet. Example: unique usernames.
- **Queue** — FIFO ordering. Implementations: LinkedList, PriorityQueue. Example: print queue.
- **Map** — Key-value pairs, unique keys. Implementations: HashMap, TreeMap. Example: phone directory.

```
List<String>   list  = new ArrayList<>();
Set<Integer>   set   = new HashSet<>();
Queue<String>  queue = new LinkedList<>();
Map<String,Integer> map = new HashMap<>();
list.add("A"); set.add(1); queue.offer("task"); map.put("score", 95);
```

Q32. Differentiate between ArrayList and Vector.

```
Feature          | ArrayList              | Vector
-----------------+------------------------+------------------------
Synchronization  | Not synchronized       | Synchronized (thread-safe)
Performance      | Faster (no sync overhead)| Slower
Growth           | Grows by 50%           | Grows by 100% (doubles)
Legacy           | Introduced in Java 1.2 | Legacy class (Java 1.0)
Iterator         | fail-fast iterator     | fail-fast + Enumeration
Preferred        | Single-threaded apps   | Multi-threaded (or use
                 |                        | Collections.synchronizedList)
```

```
ArrayList<Integer> al = new ArrayList<>();
Vector<Integer> v = new Vector<>();
al.add(10); v.add(10);
// Vector methods like addElement() are legacy
```

Q33. Differentiate between Array and Collection.

```
Feature       | Array                       | Collection
--------------+-----------------------------+-----------------------------
Size          | Fixed                       | Dynamic
Type          | Primitive + Objects         | Objects only
Performance   | Faster                      | Slightly slower
Utility       | No built-in methods         | Rich API (sort, filter, etc.)
Type safety   | Type-safe                   | Generic type-safe
Null values   | Allowed                     | Allowed (depends on type)
Memory        | Contiguous                  | Varies by implementation
```

Q34. Differentiate between throw and throws.

```
Feature       | throw                         | throws
--------------+-------------------------------+--------------------------
Purpose       | Actually throws an exception | Declares possible exceptions
Location      | Inside method body            | In method signature
Followed by   | An exception object/instance | Exception class name(s)
Number        | Only one at a time            | Multiple (comma-separated)
Type          | Checked or unchecked          | Typically checked exceptions
Required      | No (optional)                 | Required for checked exceptions
```

```
// throws: declare that method may throw
void readFile(String path) throws IOException, FileNotFoundException {
    // throw: actually throw the exception
    if(path == null)
        throw new IllegalArgumentException("Path is null");
    // ... file operations
}
```

Q35. Differentiate between final, finally and finalize keywords.

```
Keyword    | Type     | Purpose
-----------+----------+-------------------------------------------------
final      | Keyword  | Variable: constant; Method: no override;
           |          | Class: no inheritance
finally    | Block    | Always executes after try-catch, used for cleanup
           |          | (closing files, DB connections)
finalize   | Method   | Called by GC before object is destroyed;
           |          | defined in Object class; deprecated in Java 9+
```

```
final int X = 10;                    // final variable

try { int a = 10/0; }
catch(ArithmeticException e) { System.out.println("Caught"); }
finally { System.out.println("Always runs"); }  // finally block

class MyClass {
    protected void finalize() {          // finalize method
        System.out.println("GC called");
    }
}
```

Q36. Show example of multiple catch block and nested try block.

```
public class MultiCatchDemo {
    public static void main(String[] args) {
        // Multiple catch blocks
        try {
            int[] arr = new int[5];
            arr[10] = 100 / 0; // could be ArithmeticException or AIOOBE
        } catch(ArithmeticException e) {
            System.out.println("ArithmeticException: " + e.getMessage());
        } catch(ArrayIndexOutOfBoundsException e) {
            System.out.println("Array error: " + e.getMessage());
        } catch(Exception e) {
            System.out.println("General exception: " + e.getMessage());
        } finally {
            System.out.println("Multiple catch done.");
        }

        // Nested try block
        System.out.println("
--- Nested Try ---");
        try {
            System.out.println("Outer try");
            try {
                int result = 10 / 0;
            } catch(ArithmeticException e) {
                System.out.println("Inner catch: " + e.getMessage());
            }
```

```
            String s = null;
            s.length(); // NullPointerException
        } catch(NullPointerException e) {
            System.out.println("Outer catch: NullPointerException");
        }
    }
}
```

Q37. How to change priority of thread? Explain with example.

Thread priority controls which thread gets more CPU time. Range is 1 (MIN_PRIORITY) to 10 (MAX_PRIORITY), default is 5 (NORM_PRIORITY). Use setPriority() and getPriority() methods.

```
class MyThread extends Thread {
    MyThread(String name, int priority) {
        super(name);
        setPriority(priority);
    }
    public void run() {
        for(int i = 0; i < 3; i++)
            System.out.println(getName() + " (priority "
                + getPriority() + "): " + i);
    }
}

public class PriorityDemo {
    public static void main(String[] args) {
        MyThread t1 = new MyThread("LowPriority",  Thread.MIN_PRIORITY);  // 1
        MyThread t2 = new MyThread("NormPriority", Thread.NORM_PRIORITY); // 5
        MyThread t3 = new MyThread("HighPriority", Thread.MAX_PRIORITY);  // 10

        t1.start(); t2.start(); t3.start();
        // HighPriority thread likely executes more often
    }
}
```

Note: Priority is just a hint to the OS scheduler; actual behavior depends on the JVM and OS.

Q38. Explain Thread Synchronization with example.

Thread synchronization prevents multiple threads from simultaneously accessing a shared resource, avoiding **race conditions**. Use the **synchronized** keyword.

```
class BankAccount {
    private int balance = 1000;

    // synchronized method
    synchronized void withdraw(int amount) {
        if(balance >= amount) {
            System.out.println(Thread.currentThread().getName()
                + " withdrawing " + amount);
            try { Thread.sleep(100); } catch(InterruptedException e) {}
            balance -= amount;
            System.out.println("Remaining balance: " + balance);
        } else {
            System.out.println("Insufficient funds!");
        }
    }
}
```

```
public class SyncDemo {
    public static void main(String[] args) {
        BankAccount acc = new BankAccount();

        Thread t1 = new Thread(() -> acc.withdraw(700), "Thread-A");
        Thread t2 = new Thread(() -> acc.withdraw(700), "Thread-B");

        t1.start(); t2.start();
        // Without sync: both threads could pass the balance check
        // With sync: one thread completes before the other starts
    }
}
```

Q39. When should we choose abstract class and when should we choose interface?

Choose Abstract Class when:

- Classes share common code/state (fields, constructors).
- You want to provide default method implementations for some methods.
- Tight coupling is acceptable (IS-A relationship).
- You need non-public members (protected fields/methods).
- Example: Template Method Pattern — Animal with abstract sound() but concrete sleep().

Choose Interface when:

- You want to define a contract (pure behavior) with no state.
- Multiple inheritance of type is needed (a class can implement many interfaces).
- Loose coupling between unrelated classes (Serializable, Comparable, Runnable).
- Java 8+: default/static methods can provide some implementation in interfaces too.

```
// Use abstract class
abstract class Vehicle { int speed; abstract void drive(); }

// Use interface
interface Flyable { void fly(); }
interface Swimmable { void swim(); }
class Duck extends Animal implements Flyable, Swimmable { ... }
```

Q40. Differentiate between super keyword and this keyword.

```
Feature       | this                          | super
--------------+-------------------------------+---------------------------
Refers to     | Current class object          | Parent class object
Variable      | this.x accesses current field| super.x accesses parent field
Method        | this.method() current class   | super.method() parent class
Constructor   | this() calls current class    | super() calls parent class
              | constructor                   | constructor
Position      | this() must be first stmt     | super() must be first stmt
Default call  | Not implicit                  | super() called implicitly if
              |                               | no this() in constructor
```

```
class A {
    int x = 10;
    A() { System.out.println("A constructor"); }
}
class B extends A {
    int x = 20;
```

```
    B() { super(); } // calls A()
    void show() {
        System.out.println(this.x);  // 20 - current class
        System.out.println(super.x); // 10 - parent class
    }
}
```

Q41. Write a program showing multiple inheritance using extends and implements.

```
interface Flyable {
    default void fly() { System.out.println("Flying..."); }
}
interface Swimmable {
    default void swim() { System.out.println("Swimming..."); }
}
class Animal {
    void eat() { System.out.println("Eating..."); }
}

// Multiple inheritance: extends one class, implements multiple interfaces
class Duck extends Animal implements Flyable, Swimmable {
    void quack() { System.out.println("Quack!"); }
}

public class MultiInheritDemo {
    public static void main(String[] args) {
        Duck d = new Duck();
        d.eat();   // from Animal (extends)
        d.fly();   // from Flyable (implements)
        d.swim();  // from Swimmable (implements)
        d.quack(); // Duck's own method
    }
}
```

Q42. Why multiple inheritance is not supported in Java? How is it overcome?

Reason — The Diamond Problem: If two parent classes define the same method, and a child inherits from both, it's ambiguous which method to use. Java avoids this to keep the language simple and avoid runtime ambiguity.

```
// Hypothetical (NOT valid Java):
class A { void show() { System.out.println("A"); } }
class B { void show() { System.out.println("B"); } }
class C extends A, B {
    // Which show() is called? A's or B's? -- AMBIGUOUS!
}
```

Solution — Interfaces: Java allows implementing multiple interfaces. If two interfaces have the same default method, the implementing class **must** override it, resolving the ambiguity.

```
interface A { default void show() { System.out.println("A"); } }
interface B { default void show() { System.out.println("B"); } }

class C implements A, B {
    public void show() {
        A.super.show(); // explicitly choose which one
        System.out.println("C resolves ambiguity");
    }
```

```
}
```

Q43. Differentiate between overloading and overriding.

```
Feature        | Overloading                  | Overriding
---------------+------------------------------+----------------------------
Definition     | Same name, diff parameters   | Redefine parent method in child
Class          | Same class                   | Parent-child classes required
Signature      | Must differ                  | Must be identical
Return type    | Can differ                   | Must be same (or covariant)
Access modifier| Can be anything              | Cannot be more restrictive
Binding        | Compile-time (static)        | Runtime (dynamic)
Static methods | Can be overloaded            | Cannot be overridden
final methods  | Can be overloaded            | Cannot be overridden
```

```
class Calc {
    int add(int a, int b)        { return a+b; }    // overloading
    double add(double a, double b){ return a+b; }   // overloading
}

class Parent { void greet() { System.out.println("Hi from Parent"); } }
class Child extends Parent {
    void greet() { System.out.println("Hi from Child"); } // overriding
}
```

Q44. What is mutable string? Difference between String and StringBuffer.

A **mutable string** is one whose content can be changed after creation. StringBuffer (and StringBuilder) represent mutable strings.

```
Feature         | String                     | StringBuffer
----------------+----------------------------+---------------------------
Mutability      | Immutable                  | Mutable
Memory          | String Pool                | Heap
Thread safety   | Thread-safe (immutable)    | Synchronized (thread-safe)
Performance     | Slow for concatenation     | Faster than String
Method          | concat(), +                | append(), insert(), delete()
Introduced      | Java 1.0                   | Java 1.0
```

```
String s = "Hello";
s = s + " World";  // creates NEW object; old "Hello" stays in pool

StringBuffer sb = new StringBuffer("Hello");
sb.append(" World");   // modifies SAME object
sb.insert(5, ",");
sb.reverse();
System.out.println(sb); // dlroW ,olleH

// StringBuilder (Java 5+) = StringBuffer without synchronization
StringBuilder sb2 = new StringBuilder("Fast");
sb2.append("er"); // preferred for single-threaded
```

Q45. Write a program for method overloading.

```
class MathOps {
    // Overloaded add methods (same name, different parameters)
    int add(int a, int b) {
        return a + b;
    }
    double add(double a, double b) {
```

```
        return a + b;
    }
    int add(int a, int b, int c) {
        return a + b + c;
    }
    String add(String a, String b) { // concatenation
        return a + b;
    }

    // Overloaded area methods
    double area(double r) {               // circle
        return Math.PI * r * r;
    }
    double area(double l, double b) {     // rectangle
        return l * b;
    }
    double area(double b, double h, boolean triangle) { // triangle
        return 0.5 * b * h;
    }
}

public class OverloadDemo {
    public static void main(String[] args) {
        MathOps m = new MathOps();
        System.out.println(m.add(5, 3));          // 8
        System.out.println(m.add(2.5, 3.5));      // 6.0
        System.out.println(m.add(1, 2, 3));        // 6
        System.out.println(m.add("Hello ", "World")); // Hello World
        System.out.printf("Circle area: %.2f
", m.area(7));
        System.out.printf("Rect area:   %.2f
", m.area(5, 4));
    }
}
```

Q46. Explain use of inner class with example.

An inner class is a class defined inside another class. Types: Member Inner Class, Static Nested Class, Local Inner Class, Anonymous Inner Class.

Benefits: Logical grouping, access to outer class private members, cleaner code.

```
class OuterClass {
    private int data = 100;

    // 1. Member Inner Class
    class Inner {
        void show() {
            System.out.println("Outer data: " + data); // accesses private
        }
    }

    // 2. Static Nested Class
    static class StaticNested {
        void display() { System.out.println("Static nested class"); }
    }

    void method() {
        // 3. Local Inner Class
```

```
        class Local {
            void print() { System.out.println("Local inner class"); }
        }
        new Local().print();
    }
}

public class InnerDemo {
    public static void main(String[] args) {
        // Member inner class
        OuterClass outer = new OuterClass();
        OuterClass.Inner inner = outer.new Inner();
        inner.show();

        // Static nested class
        OuterClass.StaticNested sn = new OuterClass.StaticNested();
        sn.display();

        // Local class via method
        outer.method();

        // 4. Anonymous inner class (implements interface inline)
        Runnable r = new Runnable() {
            public void run() { System.out.println("Anonymous class!"); }
        };
        r.run();
    }
}
```

Q47. Write a short note on JVM.

JVM (Java Virtual Machine) is an abstract computing machine that enables Java's platform independence. It executes Java bytecode and bridges the gap between Java programs and the underlying OS/hardware.

Key Components of JVM:

- **Class Loader Subsystem:** Loads .class files (bytecode) into memory — Loading, Linking, Initialization.
- **Method Area:** Stores class metadata, static variables, method bytecode.
- **Heap:** Runtime area for all objects and instance variables (garbage collected).
- **Stack:** Each thread has its own stack; stores method frames, local variables.
- **PC Register:** Stores the address of the currently executing instruction.
- **Native Method Stack:** For native (C/C++) method calls.
- **Execution Engine:** Interprets bytecode or compiles with JIT (Just-In-Time) compiler.
- **Garbage Collector:** Automatically frees unused heap memory.

```
Java Source (.java)
     | javac
     v
Bytecode (.class)
     | JVM
     v
Machine Code (OS-specific)
```

The JVM makes Java **Write Once, Run Anywhere (WORA)** — the same .class file runs on any platform with a compatible JVM.

Q48. Explain Array of Objects with program.

(See Q24 for the concept. Here is an extended program with additional operations.)

```
class Product {
    String name;
    double price;
    int qty;

    Product(String n, double p, int q) {
        name = n; price = p; qty = q;
    }
    double totalValue() { return price * qty; }
    void display() {
        System.out.printf("%-15s Rs.%-8.2f Qty:%-5d Total: Rs.%.2f
",
            name, price, qty, totalValue());
    }
}

public class ProductInventory {
    public static void main(String[] args) {
        Product[] products = {
            new Product("Laptop",    55000, 10),
            new Product("Mouse",       450, 50),
            new Product("Keyboard",   1200, 30),
            new Product("Monitor",   12000, 15)
        };

        System.out.println("=== Product Inventory ===");
        double grandTotal = 0;
        for(Product p : products) {
            p.display();
            grandTotal += p.totalValue();
        }
        System.out.printf("
Grand Total: Rs.%.2f
", grandTotal);

        // Find most expensive product
        Product expensive = products[0];
        for(Product p : products)
            if(p.price > expensive.price) expensive = p;
        System.out.println("Most Expensive: " + expensive.name);
    }
}
```

Q49. Characteristics of OOP language.

Object-Oriented Programming (OOP) organizes software around **objects** rather than functions and logic. Java is a pure OOP language.

Four Pillars of OOP:

- **1. Encapsulation:** Bundling data (fields) and methods that operate on that data within a class. Access is controlled via access modifiers (private, public). Achieved through getters/setters. Promotes data hiding and security.
- **2. Inheritance:** A class (child) acquires properties and behaviors of another class (parent) using 'extends'. Promotes code reuse and establishes IS-A relationships. Types: single, multilevel,

hierarchical (Java doesn't support multiple class inheritance).

• **3. Polymorphism:** 'Many forms' — same method behaves differently in different contexts. Method Overloading (compile-time) and Method Overriding (runtime) are its forms.

• **4. Abstraction:** Hiding implementation details and showing only essential features. Achieved through abstract classes and interfaces.

Other OOP Characteristics:

• Classes and Objects as building blocks.

• Message Passing: Objects communicate through method calls.

• Dynamic Binding: Method calls resolved at runtime.

• Modularity: Code organized into reusable, independent modules.

Q50. Write a program using command line argument to find maximum number.

```
public class MaxFromArgs {
    public static void main(String[] args) {
        if(args.length == 0) {
            System.out.println("Usage: java MaxFromArgs num1 num2 num3 ...");
            return;
        }

        int max = Integer.parseInt(args[0]);

        for(int i = 1; i < args.length; i++) {
            try {
                int num = Integer.parseInt(args[i]);
                if(num > max) max = num;
            } catch(NumberFormatException e) {
                System.out.println("Skipping non-integer: " + args[i]);
            }
        }

        System.out.println("Numbers entered: " + java.util.Arrays.toString(args));
        System.out.println("Maximum number: " + max);
    }
}

// Compile: javac MaxFromArgs.java
// Run:     java MaxFromArgs 34 78 12 99 56 23
// Output:
//   Numbers entered: [34, 78, 12, 99, 56, 23]
//   Maximum number: 99
```